

SUPER_ADMIN Setup

Provisioning · unlock secret · password recovery — PNK Suguan System (docs/setup.md)

The system distinguishes three SUPER_ADMIN-related credentials. Mixing them up is the most common operational mistake, so here they are side by side:

Credential	What it is	Where it lives	Rotation
SUPER_ADMIN account	A user account holding the SUPER_ADMIN role. Signs in like any user; the only role that can administer other SUPER_ADMIN accounts (safeguard S3: normal admins cannot modify SUPER_ADMIN rows)	users table (argon2id hash)	Sign in !' change password, or the escape-hatch script below
Unlock secret (PNK_SUPER_ADMIN_SECRET)	Server-side secret for the Emergency Correction (ESC) Opens a temporary 30-minute correction window on ONE PUBLISHED week; week status stays PUBLISHED; every correction is audited	.env.local / secret store	Replace the value in .env.local (dev) or your secret store, then restart the app
Unlock grant (runtime)	The 30-minute window created by one successful unlock	week_unlock_grants table	Expires automatically; nothing to rotate

Provisioning a SUPER_ADMIN account

There is deliberately **no application path** that grants SUPER_ADMIN: Create/Edit User offer only ADMIN / SCHEDULER / VIEWER. SUPER_ADMIN is provisioned by an operator directly in the database (create the user with a one-time password — see `scripts/reset-super-admin.ts` for the argon2id hashing pattern — insert the `user_roles` row for the SUPER_ADMIN role, and write an audit row, `action: GRANTED_SUPER_ADMIN`).

Safeguards that remain in force: SUPER_ADMIN is never a selectable role in the UI; no code path grants it; the account is audited like any other; deactivation revokes its sessions.

The unlock secret

- Server-side only, compared timing-safe, fail-closed when unset; a mismatch returns the same generic error as a wrong role or bad state (no oracle).
- **It is not an account password** and cannot be used to sign in.
- **It is never logged, echoed, or returned to the client**; audit rows reference the unlock grant, never the secret.
- In production, supply it via your secret store, not `.env.local`.

Password recovery: the escape-hatch script

The application's Reset Password flow refuses to touch SUPER_ADMIN rows unless the actor is also a SUPER_ADMIN (safeguard S3). If the SUPER_ADMIN password is lost, use the operator script:

```
DATABASE_URL=... node node_modules/tsx/dist/cli.mjs scripts/reset-super-admin.ts --email <SUPER_ADMIN email>
```

It prints a one-time temporary password (CSPRNG, unambiguous alphabet — no 0/O/1/I/l), then:

- sets `must_change_password = true` (rotation forced at next login)
- revokes all of the account's active sessions
- writes an audited `RESET_USER_PASSWORD` row (the temporary secret is never stored or logged)

The script is **scoped**: it refuses any account that does not hold the SUPER_ADMIN role (use the `/users` UI for normal accounts) and refuses `INACTIVE` accounts (reactivate via the UI first).

Operator notes: the temporary password is shown once in the terminal — store it in a password manager immediately, and prefer rotating again after first login since it transited a terminal. The script intentionally accepts no password argument, so an operator never chooses (or leaves in shell history) another person's final password.

